COMP1009
Programming Paradigms

Lecture OO-7

Swing GUIs

This Lecture

- We will look at how to use an existing GUI class library to create GUI applications
  - To demonstrate a class library & design patterns
  - When I started this, I asked 2nd years what they wished they had seen in Java => how to create a GUI application
- We will use 'Java Swing'
  - Quite an old library
  - There are newer (and possibly better) GUI libraries
  - Swing illustrates various OO design patterns though, so useful for our learning of OO
  - Think of this as an illustration of OO rather than a brilliant class library – with added benefit of GUI

A simple Hello World GUI Program

The program should display a top level window, with a title which says “Hello World”. The main part of the window should also display the message “Hello World”. The message should be displayed in an appropriate font and the window should be just big enough for the message to be displayed. We may want to change the font later, for branding reasons, and it should be possible to do so without a lot of re-writing of the code. Pressing the close button should end the program.

- **What objects are here? (Many correct answers)**

Using nouns to help identify objects...

The program should display a **top level window**, with a **title** which says “Hello World”. The **main part of the window** should also display the **message “Hello World”.** The message should be displayed in an appropriate **font** and the window should be just big enough for the message to be displayed. We may want to change the font later, for branding reasons, and it should be possible to do so without a lot of re-writing of the code. Pressing the **close button** should end the program.

- **Some potential objects identified**

What functionality is needed?

The program should display a top level window, with a title which says “Hello World”. The main part of the window should also display the message “Hello World”. The message should be displayed in an appropriate font and the window should be just big enough for the message to be displayed. We may want to change the font later, for branding reasons, and it should be possible to do so without a lot of re-writing of the code. Pressing the close button should end the program.

- **Identify the functionality – operations done**

Using verbs to identify functionality

The program should **display** a top level window, with a title which **says** “Hello World”. The main part of the window should also **display** the message “Hello World”. The message should be **displayed in an appropriate font** and the window should **be just big enough for the message** to be displayed. We may want to **change the font** later, for branding reasons, and it should be possible to do so without a lot of re-writing of the code. **Pressing** the close button should **end** the program.

- **Some example operations/responsibilities**

Class libraries

- We will often have a library of classes available
  - Libraries are sets of classes which usually work together
  - Understanding what classes are available is useful
- Sometimes we can just re-use existing classes to make our program
- You can make objects of a class type
  - You use the new operator to create new objects
- Good objects will be adaptable in some ways
  - You usually change their *state* – change the data in them
  - E.g. the title of a window
- (We will use inheritance later for more complex changes)
- Sometimes the class library may point us towards certain object types (i.e. classes) and structures
  - E.g. window class could include title bar and close button

OO Models vs ER diagrams

- **You are also currently considering ER-diagrams**
- There are big differences in what we are trying to do:
  - In ER diagrams we care about the data, how to group it and how it is related to other data
  - In OO class diagrams we care about **what is done** and **what is doing it**, i.e. objects and operations
  - This may mean that we group objects differently, create new objects just to do some behaviour, or split objects which have multiple behaviours
  - Design patterns (example solutions for tasks) will help us to know when to do this until we have become experts

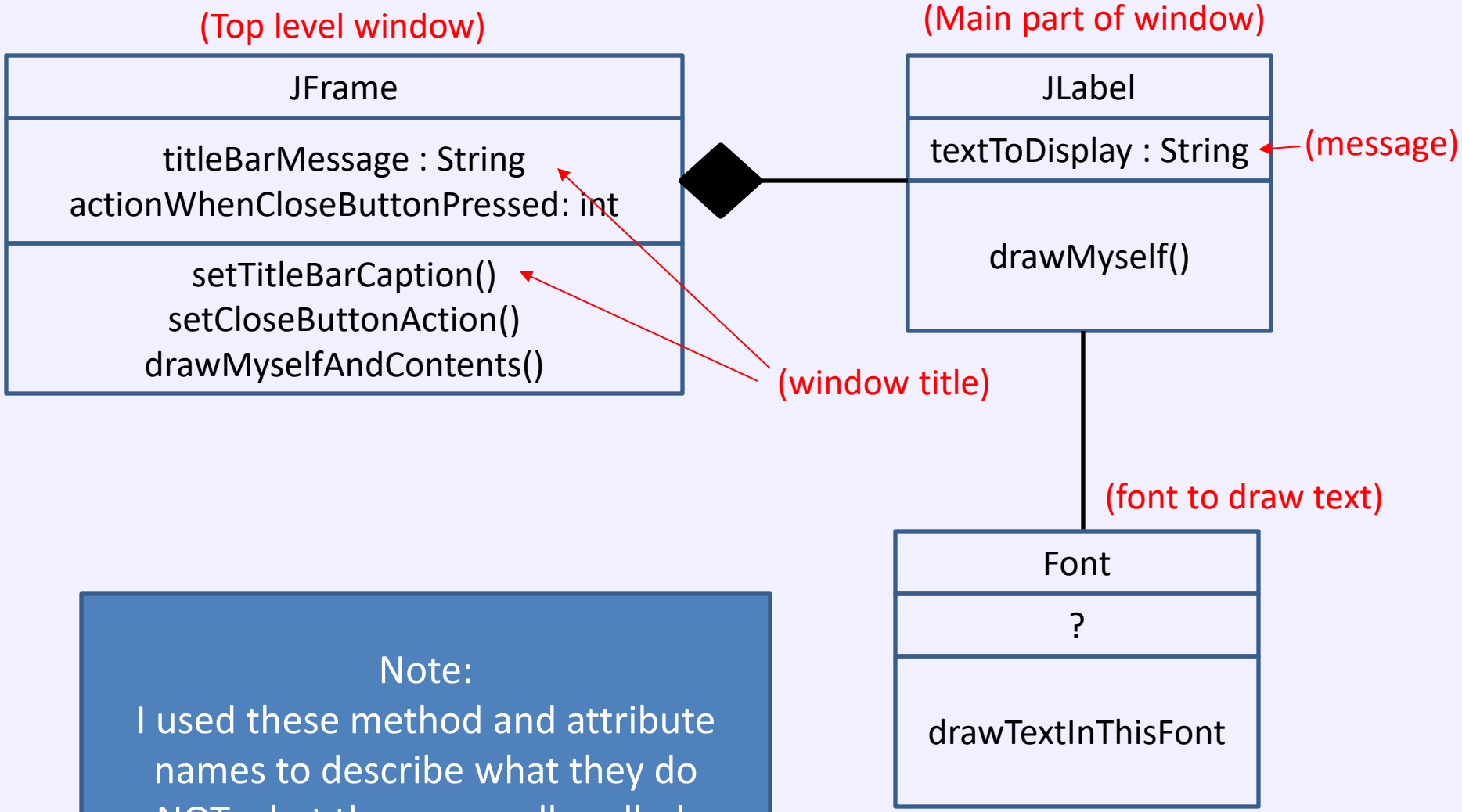
Java Swing

- A class library provides a set of classes for use
  - Create objects of existing classes to solve tasks
  - Configure objects by setting state (using methods)
  - Extend using sub-classing to create new classes
- If we have a class library we will usually start by looking at **what it can do** and **fit our plans around it**
  - How is it intended to be used? Use it that way
- The “Java Swing” class library provides a lot of ‘base classes’ that we can create objects from, or extend to make our own classes to alter behaviour
  - In the coursework I will give you variants of these to use

Some Java Swing User Interface Classes

- Top level window : JFrame
  - Close button, minimise button, maximise button
  - Frame, resizable?
- Controls/components
  - Button : JButton
  - Label (displays text): JLabel
  - Text box : JTextField
  - List box : JListbox
  - Etc….
- Other useful classes
  - Font – to draw the text with
  - Layout Managers – refine how it appears

Example Class Diagram



Example of basic GUI Hello World

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class GUIHelloWorld  { // Just the one class
    public static void main(String[] args)  { // Same main function as before – static
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Close window exits
        guiFrame.setTitle("Hello World!"); // Set a caption/title bar content for the frame
        guiFrame.setLocationRelativeTo(null);  // centre on screen
        guiFrame.setLayout( new FlowLayout() ); // Layout : how to layout components
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!"); // Set its message to show
        label.setFont(new Font("Ariel", Font.BOLD, 30)); // Set a font for the label
        guiFrame.add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

Divided into sections

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class GUIHelloWorld  {
    public static void main(String[] args)  {
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        guiFrame.setTitle("Hello World!");
        guiFrame.setLocationRelativeTo(null);
        guiFrame.setLayout( new FlowLayout() );
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!");
        label.setFont(new Font("Ariel", Font.BOLD, 30));
        guiFrame.add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

import

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;
```

Import just tells the compiler which package each class is in.
e.g. when I say 'Font' I mean the "java.awt.Font" class
Packages are structured like a filesystem of directories

```
public class GUIHelloWorld  {
    public static void main(String[] args)  {
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        guiFrame.setTitle("Hello World!");
        guiFrame.setLocationRelativeTo(null);
        guiFrame.setLayout( new FlowLayout() );
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!");
        label.setFont(new Font("Ariel", Font.BOLD, 30));
        guiFrame.add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

New objects are created

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class GUIHelloWorld  {
    public static void main(String[] args)  {
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        guiFrame.setTitle("Hello World!");
        guiFrame.setLocationRelativeTo(null);
        guiFrame.setLayout( new FlowLayout() ); // Create a new FlowLayout object
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!");
        label.setFont(new Font("Ariel", Font.BOLD, 30)); // Create a new Font object
        guiFrame.add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

new is used to create objects
FOUR new objects are created here

Customising objects (not classes)

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class GUIHelloWorld  {
    public static void main(String[] args)  {
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        guiFrame.setTitle("Hello World!");
        guiFrame.setLocationRelativeTo(null);
        guiFrame.setLayout( new FlowLayout() );
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!");
        label.setFont(new Font("Ariel", Font.BOLD, 30));
        guiFrame.add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

Customise objects by calling methods on them

Customise the JFrame object we created

Customise the JLabel object we created

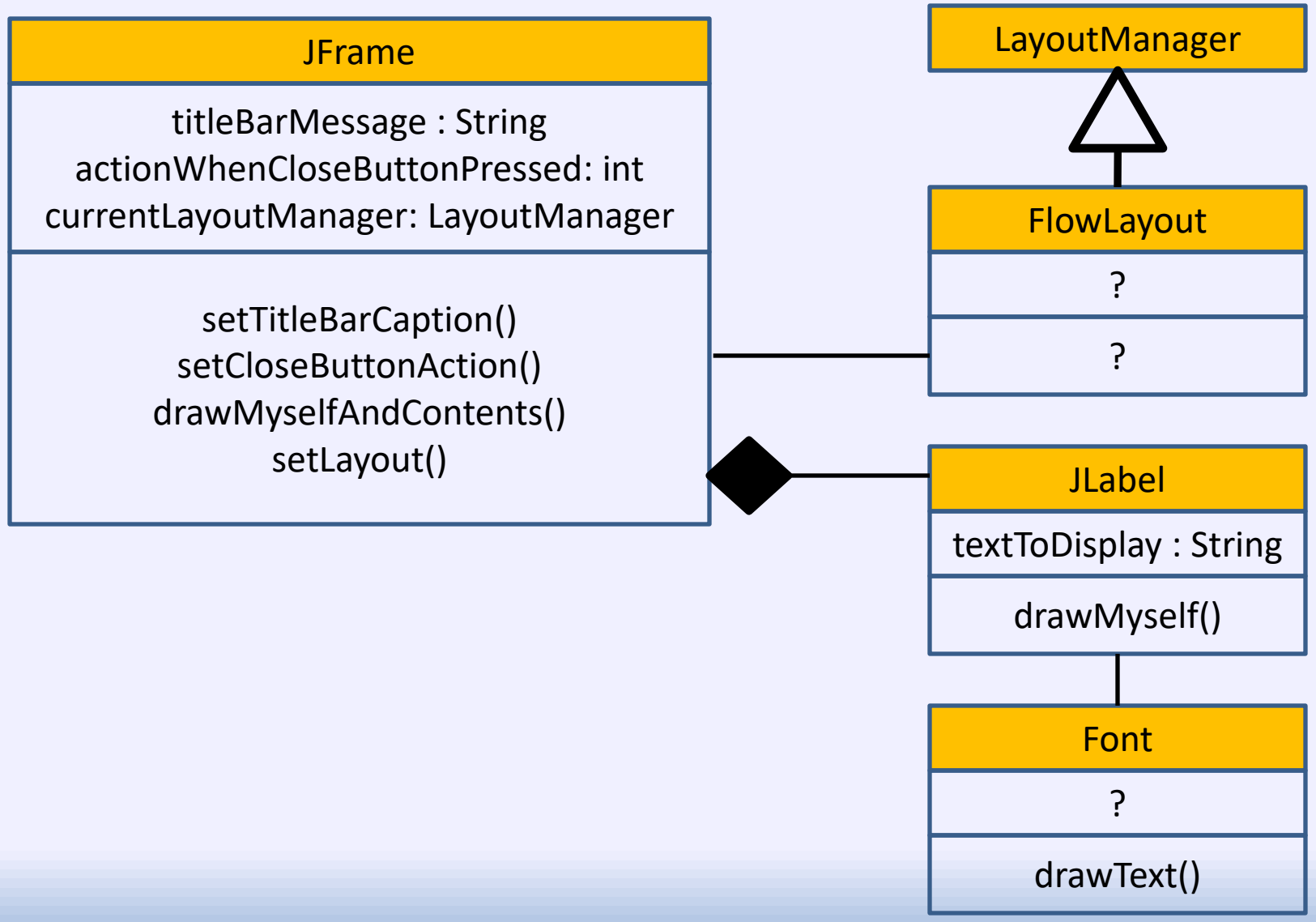
Resizing and showing the window

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class GUIHelloWorld  {
    public static void main(String[] args)  {
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        guiFrame.setTitle("Hello World!");
        guiFrame.setLocationRelativeTo(null);
        guiFrame.setLayout( new FlowLayout() );
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!");
        label.setFont(new Font("Ariel", Font.BOLD, 30));
        guiFrame.add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

Overall method…

```
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class GUIHelloWorld  { // Just the one class
    public static void main(String[] args)  { // Same main function as before – static
        JFrame guiFrame = new JFrame(); // Create a new frame object – top level window
        guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Close window exits
        guiFrame.setTitle("Hello World!"); // Set a caption/title bar content for the frame
        guiFrame.setLocationRelativeTo(null);  // centre on screen
        guiFrame.setLayout( new FlowLayout() ); // Layout : how to layout components
        JLabel label = new JLabel();  // Create a new label object
        label.setText("Hello World (again)!"); // Set its message to show
        label.setFont(new Font("Ariel", Font.BOLD, 30)); // Set a font for the label
        guiFrame.add(label); // Add the label to the frame, so that it will show
        guiFrame.pack(); // Resize frame to fit content
        guiFrame.setVisible(true); // Display it – until you do it will not appear
    }
}
```

Class diagram with a bit more detail…



Note: we should actually do this…

```
JFrame guiframe = new JFrame();
guiframe.getContentPane().setLayout( new FlowLayout() );

JLabel label = new JLabel("Hello World");
label.setFont( new Font("Ariel", Font.BOLD, 60) );
guiframe.getContentPane().add(label);

guiframe.setTitle("Hello World");
guiframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
guiframe.pack();
guiframe.setVisible(true);
```

Note: I added directly to the frame to simplify matters. In fact, the frame already has components inside it, and strictly speaking we should add to the content pane of the frame so that we don't break any other behaviour later

Demo: Working with labels and objects

Starting GUI Basic Class

```
package pgp;                    // Choose a package
import java.awt.FlowLayout; // Remember ctrl-shift-o in Eclipse to generate these
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class MyGUIBasicClass
{
  public static void main(String[] args)
  {
    JFrame guiFrame = new JFrame();
    guiFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    guiFrame.setTitle("Hello World!");
    guiFrame.setLocationRelativeTo(null);
    guiFrame.getContentPane().setLayout( new FlowLayout() );
    JLabel label = new JLabel();
    label.setText("Hello World (again)!");
    label.setFont(new Font("Ariel", Font.BOLD, 30));
    guiFrame.getContentPane().add(label);
    guiFrame.pack(); // Resize frame to fit content
    guiFrame.setVisible(true);        }
}
```

Object references are not objects

- Object references store references to objects
  - A way to refer to an object (a pointer)
- Objects are created using new
- You can split the two up…

```
public static JLabel createLabel()
{
      return new JLabel();
}

JLabel label = createLabel(); // Create a new label object
```

Customisation can be moved to method…

```
public static JLabel createLabel()
{
    JLabel label = new JLabel();
    label.setText("Hello World (again)!");
    label.setFont(new Font("Ariel", Font.BOLD, 30));
    return label;
}
```

- And we could then add two labels very easily

```
//JLabel label = createLabel(); // Create a new label
guiFrame.getContentPane().add(createLabel());
guiFrame.getContentPane().add(createLabel());
```

Using a static counter

```
public static JLabel createLabel()
{
   JLabel label = new JLabel();
   label.setText("Label" + labelCount++);
   label.setFont(
         new Font("Ariel", Font.BOLD, 30));
   return label;
}

public static int labelCount = 1;
```

- Static variables are NOT in objects – they are shared
- Static methods are NOT associated with a specific object

JLabel Constructors

- Remember: When you create an object a constructor gets called
  - A function with the same name as the class and no return type
- Classes can have multiple constructors
- We can pass extra values to constructors
  - We put these between the ()
  - The parameter types that we use determine which constructor gets called
- **JLabel has a constructor, which takes a string to display**
  - **So we don't actually need to call setText()**

- i.e. You can use either:

```
JLabel label = new JLabel();
label.setText("Hello World (again)!");
```

- Or:

```
JLabel label = new JLabel("Hello World (again)!");
```

- Online documentation will show these for many classes if eclipse autocomplete does not
- More later on JLabel constructors…

We could create an object (or two)

- At the moment we have no object for the class with our main in – so we have to have the code in a static method
  - Remember: Static methods are not associated with an object
  - We could make an object though
  - Then do the work in the constructor instead, for example

```
public static void main(String[] args)
{
      new MyGUIBasicClass ();
}

public MyGUIBasicClass () // Constructor
{
      …
}
```

Overview/summary of OO

1. Identify the classes you need
   - Understand your class library if you are using one
2. Create the objects – using existing or new classes
3. Customise the objects
   - Creating any missing objects that you need for this
4. Call operations on the objects to get them to do what you need them to do

- In OO programming you are usually:
  - Using existing classes (so many Java classes online!)
  - Creating or adapting classes (when flexibility insufficient)
    - Aggregation or inheritance
  - Creating and configuring objects of these classes
  - Plugging the objects together
- Reduces code, reduces debugging, shares the workload

Next Lecture

- Lecture 8 (Tuesday): Custom (coloured) labels
  - Inheritance example
  - Layout managers
    - Example of strategy pattern
- Lecture 9 (Friday): Buttons
  - Interfaces and abstract classes
  - Button handlers
    - Example of observer design pattern
- Lab 4: an example of creating a user interface
- Lecture introducing CW2c/2d (Tuesday 3rd March)
  - Main coursework is in two parts, one is the user interface